BÁO CÁO THỰC HÀNH LAB 17

MERGE CONFLICT

Người báo cáo: Liamz07

1. Mục tiêu:

- **Hiểu nguyên nhân:** Bản chất của hiện tượng xung đột dữ liệu khi hai hay nhiều nhánh cùng chỉnh sửa một dòng mã nguồn trên cùng một tệp tin.
- **Kỹ thuật giả lập:** Chủ động tạo ra Merge Conflict giữa hai nhánh làm việc song song để phục vụ kiểm thử và nâng cao kỹ năng gỡ lỗi.
- **Quy trình gỡ lỗi (Resolution Workflow):** Thực hiện đọc hiểu thẻ đánh dấu xung đột (<<<<<<<, =======, >>>>>>>), giải quyết mâu thuẫn dữ liệu bằng tay (manual resolution), sau đó đóng gói phiên bản bằng chuỗi lệnh add và commit.

2. Môi trường và chuẩn bị:

- **Hệ điều hành:** Ubuntu Linux / macOS / Windows (sử dụng Terminal / Git Bash)
- **Công cụ hỗ trợ:** Git CLI, trình biên soạn văn bản (Nano / VS Code).

3. Nội dung thực hành chi tiết:

BÀI 1: KHỞI TẠO KHO CHỨA VÀ TỆP TIN MỐC

- **Mục đích:** Đặt nền móng cho kho chứa và tạo một tệp dữ liệu chung trên nhánh main.
- **Thao tác thực thi:**

```
mkdir git-conflict-lab
cd git-conflict-lab
git init

# Tạo tệp tin ban đầu
echo "Original content on main branch" > config.txt
git add config.txt
git commit -m "Initial commit with config.txt"
```

- **Kết quả:** Nhánh main đã lưu bản commit đầu tiên chứa tệp config.txt.

BÀI 2: TẠO NHÁNH MỚI VÀ CHỈNH SỬA DÒNG DỮ LIỆU MỤC TIÊU

- **Mục đích:** Giả lập công việc của Lập trình viên A trên nhánh feature/update-config.
- **Thao tác thực thi:**

```
# Tạo và chuyển sang nhánh develop
git checkout -b develop

# Tạo tệp cấu hình nền tảng
echo "console.log('App init');" > app.js
git add app.js
git commit -m "Setup base application logic"
```

BÀI 3: CHỈNH SỬA ĐÈ LÊN CÙNG DÒNG DỮ LIỆU TẠI NHÁNH MAIN (GIẢ LẬP CONFLICT)

- **Mục đích:** Giả lập công việc của Lập trình viên B chỉnh sửa cùng dòng dữ liệu trên nhánh main.
- **Thao tác thực thi:**

```
# Tách nhánh từ develop
git checkout develop
git checkout -b feature/login

# Viết mã nguồn tính năng Login
echo "function login(user, pass) { return true; }" > login.js
git add login.js
git commit -m "Add login functionality"
```

- **Kết quả:** Cả hai nhánh main và feature/update-config đều đã tiến lên 1 commit mới, chỉnh sửa cùng một dòng trong tệp config.txt với nội dung hoàn toàn khác nhau.

BÀI 4: THỰC THI MERGE VÀ GHI NHẬN LỖI XUNG ĐỘT (CONFLICT TRIGGER)

- **Mục đích:** Thực hiện lệnh git merge để kích hoạt xung đột hệ thống.
- **Thao tác thực thi:**

```
# Đảm bảo đang đứng ở nhánh main
git checkout main

# Hợp nhất nhánh feature/update-config vào main
git merge feature/update-config
```

- **Kết quả hiển thị:**

```
Auto-merging config.txt
CONFLICT (content): Merge conflict in config.txt
Automatic merge failed; fix conflicts and then commit the result.
```

- **Giải thích nguyên nhân:**
  - *Cơ chế hợp nhất 3 chiều:* Khi thực hiện lệnh git merge, Git sẽ tìm điểm chung gần nhất của hai nhánh (gọi là Common Ancestor — ở đây là commit khởi tạo chứa dòng "Original content on main branch") và đối chiếu với trạng thái mới nhất của cả hai nhánh (main và feature/update-config).
  - *Sự mâu thuẫn không thể tự động quyết định:* Cả hai nhánh đều đã thay đổi cùng một vị trí (dòng số 1 của tệp config.txt) so với bản gốc ban đầu, nhưng lại đưa ra hai nội dung hoàn toàn trái ngược nhau ("Content modified by Main branch" và "Content modified by Feature branch").
  - *Nguyên tắc bảo toàn dữ liệu của Git:* Vì Git chỉ là công cụ quản lý phiên bản, nó không có khả năng tự suy đoán logic nghiệp vụ xem đoạn mã nào mới là đúng hay quan trọng hơn. Nếu tự ý chọn một bên, dữ liệu của bên còn lại sẽ bị ghi đè và mất vĩnh viễn. Do đó, Git bắt buộc phải tạm dừng tiến trình hợp nhất (Automatic merge failed), đánh dấu vị trí mâu thuẫn trực tiếp trong tệp tin và yêu cầu con người can thiệp quyết định.

BÀI 5: GIẢI QUYẾT XUNG ĐỘT VÀ ĐÓNG GÓI COMMIT (RESOLVE → ADD → COMMIT)

- **Mục đích:** So sánh chi tiết thay đổi giữa Working Directory và Staging Area trước khi tạo commit mới.

BÀI 5.1: MỞ TỆP TIN KIỂM TRA DẤU VẾT XUNG ĐỘT (RESOLVE)

- **Thao tác:** Mở tệp config.txt bằng trình biên soạn (ví dụ: cat config.txt hoặc nano config.txt).
- **Nội dung thẻ đánh dấu:**

```
<<<<<<< HEAD
Content modified by Main branch
=======
Content modified by Feature branch
>>>>>>> feature/update-config
```

- **Thao tác xử lý bằng tay (Manual Resolution):** Tiến hành xóa các thẻ đánh dấu (<<<<<<<, =======, >>>>>>>) và thống nhất giữ lại nội dung hợp nhất hoàn chỉnh:

```
Content modified by Main branch & Feature branch (Resolved)
```

BÀI 5.2: ĐƯA TỆP ĐÃ SỬA VÀO STAGING AREA (ADD)

- **Thao tác thực thi:** git add config.txt
- **Phân tích:** Lệnh git add báo cho Git biết rằng xung đột tại tệp config.txt đã được giải quyết xong.

BÀI 5.3: HOÀN TẤT QUÁ TRÌNH HỢP NHẤT (COMMIT)

- **Thao tác thực thi:** git commit -m "Fix merge conflict between main and feature/update-config"

BÀI 6: KIỂM TRA LỊCH SỬ SAU KHI XỬ LÝ XUNG ĐỘT

- **Thao tác thực thi:** git log --graph --oneline
- **Phân tích kết quả:** Cây lịch sử hiển thị điểm hội tụ (Merge Commit) kết nối hai nhánh lại với nhau an toàn.

4. Quy trình xử lý conflict tổng hợp:

Bước	Lệnh / Thao tác	Mô tả chi tiết
Resolve	Mở file bị contact	Xóa các ký tự <<<<<<<, =======, >>>>>>> và thống nhất nội dung mã nguồn cuối cùng.
Add	git add <file_path>	Đánh dấu tệp tin đã giải quyết xong xung đột vào vùng chờ (Staging Area).
Commit	git commit -m “message”	Ghi nhận commit hợp nhất hoàn chỉnh vào kho lưu trữ (Repository).

5. Kết luận và bài học rút ra:

- **Kết quả đạt được:** Thành thạo quy trình tự tạo Merge Conflict, hiểu rõ cơ chế 3-way merge dẫn đến xung đột, phân tích ký hiệu nhận diện của Git và hoàn thành chuỗi thao tác khắc phục Resolve → Add → Commit.
- **Kỹ năng tích lũy:** Rèn luyện phản xạ bình tĩnh xử lý khi gặp xung đột mã nguồn trong các dự án làm việc nhóm (Teamwork), không hoảng loạn dùng các lệnh xóa bạo lực mà chủ động rà soát dòng code mâu thuẫn để đưa ra quyết định tích hợp chính xác.

HẾT!